

VERISPRINT

AI-Verified Engineering Visibility for Non-Technical Teams

Product Requirements Proposal (PRP) · Product Requirements Document (PRD)
Workflow & System Flow · Production Architecture · Market Viability · Brand Naming

Prepared: August 2026
Confidential working document — internal planning use.

Table of Contents

- 1. Product Requirements Proposal (PRP)
- 2. Market & Competitive Analysis
- 3. Product Requirements Document (PRD)
- 4. User Workflow & System Flow
- 5. Production Architecture
- 6. Business Viability — Can This Make Money Fast?
- 7. Go-to-Market & Pricing
- 8. Risks & Mitigations
- 9. Brand Name Recommendations

1. Product Requirements Proposal (PRP)

Problem Statement

Non-technical stakeholders — product managers, founders, clients, and executives — cannot verify engineering progress on their own. They depend entirely on what a developer reports in standup, Slack, or a status doc. This creates a trust gap: work can be reported as 'done' or 'in progress' when the underlying code tells a different story — incomplete, stubbed, untested, or not actually wired into the product. Today, closing that gap requires a technical person to manually read diffs and translate them, which doesn't scale and rarely happens consistently.

Opportunity

A software layer that connects directly to GitHub, reads every commit and pull request in depth, and produces a plain-language, verified account of what was actually built — then reconciles that against what was claimed in standups or tickets. This does not replace a developer's judgment; it gives non-coders an honest, always-on translator for the codebase.

Proposed Solution

A GitHub-connected platform (codename: VeriSprint) that ingests commits and PR diffs, runs them through an LLM analysis pipeline that reasons about functional completeness (not just line counts), and produces:

- A plain-English breakdown of what each commit/PR actually does, for a non-technical reader.
- A completion-confidence score per task, based on evidence in the code (tests, dead code, TODOs, wiring).
- A 'claimed vs shipped' reconciliation between standup/Slack updates and actual commits.
- Rollups for stakeholders: daily, sprint, and per-person views.

Why Now

LLMs have only recently become good and cheap enough to reliably reason about diffs in context rather than just summarizing commit messages. Most existing tools either summarize surface-level text (commit messages) or report abstract metrics (cycle time, DORA) without judging whether the work is functionally real. That gap is the wedge.

2. Market & Competitive Analysis

This space is not empty. It splits into three tiers, and each leaves the specific problem you described — verifying standup claims against real code — largely unsolved.

Tier 1: Enterprise Engineering Intelligence Platforms

Jellyfish, LinearB, Swarmia, GitClear, and Allstacks aggregate Git, CI/CD, and ticket data into leadership-facing dashboards (cycle time, DORA metrics, investment allocation). They are built for VPs of Engineering reporting to the C-suite, priced per-seat (\$15–35/dev/month, or \$100K+/year at enterprise scale for Jellyfish), and require a sales-led onboarding. They report patterns and velocity — they do not read a diff and tell a PM 'this feature is not actually finished.'

Tier 2: Commit/PR Summarizers

Tools like whatthediff.ai, the gpt-commit-summarizer GitHub Action, and DigestAI generate plain-language summaries of individual commits or PRs. They're lightweight and cheap, but purely descriptive — they tell you what changed, not whether it's actually complete, tested, or matches what was promised.

Tier 3: Surveillance-Style Productivity Scorers

Tools like Codemetrics analyze commit activity to rank and badge developers. These generate real backlash from engineering teams because they feel like a lie detector aimed at individuals, and developers learn to game them.

The Gap You Can Own

Nobody in this landscape does reconciliation: comparing what a person said in standup against what the code actually shows, with a completion-confidence judgment grounded in the diff (tests, dead code, wiring), delivered specifically for non-technical PMs and founders on small-to-mid teams — not enterprise VPs. That is a narrower, sharper, and cheaper-to-build wedge than the incumbents.

Tier	Examples	Strength	Gap it leaves open
Enterprise SEI	Jellyfish, LinearB, Swarmia	Deep metrics, exec reporting	Expensive, sales-led, no per-commit truth check
Summarizers	whatthediff.ai, DigestAI	Cheap, fast, plain-language	Descriptive only, no completion judgment
Productivity scorers	Codemetrics	Activity visibility	Feels punitive, gets gamed, team backlash
VeriSprint (proposed)	—	Claim-vs-code verification + non-coder friendly	Needs to prove judgment accuracy

3. Product Requirements Document (PRD)

Vision

Give every non-technical stakeholder an honest, always-current translator between the codebase and the business — so 'I've done this' in standup can be checked in seconds, not assumed.

Target Users

- **Primary:** Product managers and founders at small-to-mid engineering teams (5–50 developers) who are non-technical or semi-technical.
- **Secondary:** Engineering managers who want an honest second opinion without manually reviewing every diff.
- **Tertiary:** Agencies and freelance dev shops who need to prove progress to non-technical clients.

Core Feature Set

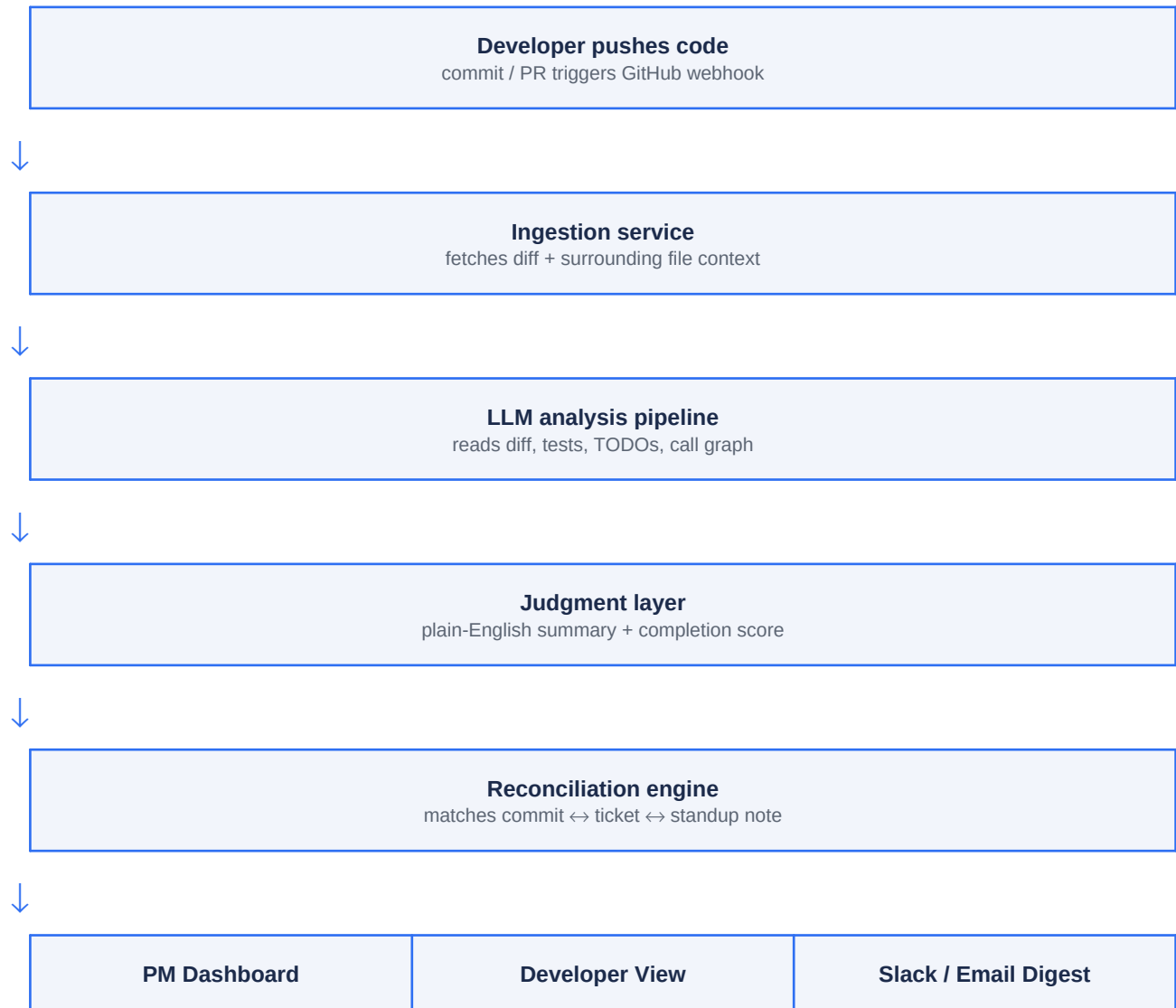
Feature	Description
Deep commit/PR analysis	LLM reads the diff in file context (not in isolation) and explains functional changes in plain language.
Completion-confidence score	Score per task/ticket based on evidence: tests added/passing, dead code, TODOs, hardcoded stubs, whether new code is actually called.
Claimed vs Shipped reconciliation	Pulls standup notes / Slack updates and automatically compares them against real commits for that period; flags mismatches.
Plain-English changelog	Auto-generated, non-technical changelog per PR/sprint for stakeholder updates.
Mismatch alerts	Notifies PM when a person's commits don't match their assigned ticket, or when a ticket has no matching commits at all.
Standup draft assist	Drafts the developer's own standup update from their real commits, for them to review/edit — reduces friction and dishonesty risk.
Blocker/risk detection	Flags stalled work: repeated small commits to the same broken function, reverted commits, long-open draft PRs.
Sprint rollup report	Stakeholder-facing report of what actually shipped vs what was planned, auto-generated at sprint end.
Cross-file impact map	Simple visual showing which parts of the product (not code) a change touches, for non-technical readers.
Historical accuracy score	Per-person, over time: how often stated progress matched actual shipped code — framed as calibration, not punishment.
Two-way transparency	Developers see the same dashboard as PMs, and can annotate/contest an AI judgment directly.

Non-Functional Requirements

- Read-only GitHub access by default (GitHub App with least-privilege scopes) — no write access needed for MVP.
- Analysis latency: summarize a typical PR within 60 seconds of merge/push.
- Data residency: customer code/diffs must not be used to train third-party models; clear data retention policy.
- Explainability: every AI judgment must cite the specific evidence (file, line range, test result) it used.
- Framing safeguard: outputs are phrased as questions/flags, not verdicts, to avoid an adversarial tool culture.

4. User Workflow & System Flow

End-to-End Flow



Daily Cycle (Stakeholder Perspective)

- Developer commits/pushes throughout the day as normal — no extra work required.
- System silently analyzes each commit/PR as it lands.
- Before standup, the developer gets an auto-drafted summary of their own real progress, for them to confirm or edit.
- PM opens the dashboard: sees claimed vs shipped side by side, completion-confidence per ticket, and any flagged mismatches.
- At sprint end, an automatic stakeholder-facing rollup report is generated for wider distribution.

Mismatch Handling Flow

- System detects a gap (e.g., ticket marked 'in review' has zero related commits in 3 days).
- Flag is raised as a question, not an accusation, visible to both developer and PM.
- Developer can respond/annotate directly in the tool (e.g., 'blocked on API access', 'covered in ticket #482').
- If unresolved, it surfaces in the PM's daily digest for a real conversation — not an automated penalty.

5. Production Architecture

High-Level Components



Component Detail

Layer	Technology direction	Purpose
Integration	GitHub App (webhooks + REST/GraphQL API), later GitLab/Bitbucket, Jira/Linear, Slack	Read-only ingestion of commits, PRs, tickets, and standup messages.
Queue	Managed queue (SQS/PubSub) or Kafka at scale	Decouples ingestion spikes from analysis workers; enables retries.
Ingestion workers	Stateless workers (Node/Python)	Fetch diffs, resolve surrounding file context, detect test/TODO signals.
LLM orchestration	Claude via API, with tool-calling for selective deep-dives into specific files	Reasons about functional completeness; avoids re-reading whole repos per commit to control cost.
Data store	Postgres (structured), object storage (raw diffs/artifacts), vector store (semantic code search)	Persistence, historical scoring, and fast retrieval for the reconciliation engine.
Reconciliation engine	Rules + LLM matching service	Aligns commits ↔ tickets ↔ standup text; computes claimed-vs-shipped deltas.
API layer	REST/GraphQL, authenticated per workspace	Serves dashboard and third-party integrations (Slack bot, email digest).
Frontend	React web app	Two views: PM/stakeholder dashboard and developer self-view (same data, different framing).
Notifications	Slack app + email digest service	Delivers daily/sprint rollups and real-time mismatch flags.
Security	SOC 2-track from day one; encrypted at rest/in transit; no training on customer code	Required to sell into any team handling proprietary codebases.

Cost Control Note

Full-repo LLM analysis on every commit is expensive at scale. The orchestration layer should use a selective, agentic approach: first classify a commit's scope cheaply, then only pull in deep file context and run the expensive reasoning pass for commits tied to active, tracked tickets — not every micro-commit in the repo.

6. Business Viability — Can This Make Money Fast?

Honest Assessment

This is a credible SaaS idea with a real, specific pain point and a defensible wedge against existing players. It is **not**, however, a 'fast money' idea in the sense of quick, low-effort revenue. Realistic expectations:

- **Sales cycle:** Selling into teams (even small ones) means convincing a PM and getting a codebase connected — this is B2B SaaS with a multi-week trial cycle, not an instant-purchase consumer product.
- **Trust is the real product**, and trust takes time to earn: the AI's judgment must be visibly accurate before teams will let it influence standup conversations. Early inaccurate judgments will kill adoption fast.
- **Crowded-adjacent market:** Buyers already fund Jellyfish/LinearB-type budgets; you're not creating a new budget line, you're competing for a slice of an existing 'engineering visibility' spend — which helps credibility but means informed buyers will compare you to funded incumbents.
- **Cultural resistance risk:** If it's perceived as a surveillance tool, engineering teams will resist or sabotage adoption — this is the single biggest threat to revenue velocity, bigger than the tech risk.

Where the Fast-Money Potential Actually Is

- **Low technical barrier to an MVP:** GitHub API + LLM diff analysis is buildable by one person in weeks, not months — so time-to-first-paying-customer can be fast even if the sales cycle per customer is not instant.
- **Cheap distribution wedge:** a free 'standup summary generator' (auto-draft what you did today, from real commits) can be given away to individual developers as a viral top-of-funnel, then upsell the PM-facing reconciliation dashboard as the paid tier.
- **Underserved segment:** small-to-mid teams priced out of Jellyfish's enterprise contracts are actively looking for something cheaper and faster to set up — this is a real, currently-underserved buyer.

Verdict

Good idea, moderate-speed money, not fast money. Expect a working paid MVP in 4–8 weeks, first paying customers within 2–3 months if you nail the free-tool-to-paid-tier funnel, and meaningful recurring revenue only after the accuracy and trust bar is cleared — typically 6–12 months in B2B dev-tools SaaS. The idea is strong enough to justify building; the money is 'realistic and compounding,' not 'fast.'

7. Go-to-Market & Pricing

Suggested Launch Sequence

- **Phase 1 (weeks 1–4):** Free CLI/GitHub Action that generates a plain-English standup summary from a developer's real commits. No dashboard, no reconciliation — pure goodwill + distribution.
- **Phase 2 (weeks 5–10):** Paid PM dashboard: claimed-vs-shipped view, completion-confidence scoring, mismatch alerts. Sell directly to PMs/founders who used the free tool.
- **Phase 3 (months 4+):** Sprint rollups, Slack/Jira deep integration, historical accuracy scoring, team-health framing to expand upmarket without becoming a surveillance product.

Pricing Direction

Tier	Price direction	Includes
Free	\$0	Standup summary generator for individual developers, GitHub Action.
Team	\$12–20 / developer / month	Full dashboard, reconciliation, mismatch alerts, Slack digest.
Growth	\$25–35 / developer / month	Sprint rollups, historical accuracy trends, Jira/Linear deep sync.

This undercuts Jellyfish's enterprise pricing and sits close to Swarmia/LinearB's per-seat range, while offering a sharper, more specific feature (claim verification) that those tools don't do.

8. Risks & Mitigations

Risk	Mitigation
AI judgment is wrong or unfair to a developer	Always show evidence with the verdict; let developers annotate/contest; frame as questions not accusations.
Perceived as a surveillance/snitching tool	Two-way transparency by default; developer gets the same dashboard; market as 'visibility for PMs,' never 'catch liars.'
LLM analysis cost scales badly with repo size/activity	Selective deep-dive only on ticket-linked commits; cheap pre-classification pass first.
Incumbents (LinearB, Jellyfish) add a similar feature	Move fast on the narrow wedge; stay cheaper and faster to set up than enterprise-sales competitors.
Data security concerns block enterprise-adjacent buyers	Read-only scopes, no training on customer code, SOC 2 roadmap from day one.
Low trial-to-paid conversion if accuracy is weak early	Launch the free standup-summary tool first to prove usefulness before charging for judgment features.

9. Brand Name Recommendations

Names below lean toward 'truth/verification + engineering' without sounding punitive. Check domain and trademark availability before committing — these are directional suggestions, not cleared names.

Name	Why it works
VeriSprint	'Verify' + 'sprint' — ties directly to agile workflow language PMs already use.
ShipTruth	Plain, memorable, directly names the core value: knowing what actually shipped.
CodeHonest	Very literal, easy to explain in one line, slightly playful.
Commitly	Pun on 'commit' + friendly SaaS-style suffix; easy to say and brand.
ClearCommit	Suggests transparency and clarity around every commit; clean, professional tone.
Standclear	Blend of 'standup' + 'clarity' — ties to the standup-verification use case specifically.
DiffSense	Technical but approachable; signals it 'makes sense of diffs' for non-coders.
ProofPush	Ties 'proof' to 'git push' — developer-friendly wordplay, still clear to non-coders.
Groundtruth.dev	Borrows the ML term 'ground truth' (the verified real answer) — resonates with technical buyers, still explainable to non-tech ones.
Actuallydone.io	Cheeky, very literal, highly memorable; good for the free viral top-of-funnel tool specifically.

Recommendation

VeriSprint or **ClearCommit** for the main paid product (professional, agency/enterprise-credible), and **Actuallydone.io** as a fun, separate brand for the free standup-summary tool used as the top-of-funnel wedge — a common SaaS pattern (free playful tool feeding a serious paid platform).

End of document.